

A Tight Lower Bound on the TdScript Trapdoor Memory-Hard Function

Jeremiah Blocki¹  and Seunghoon Lee^{a,2} 

¹ Purdue University, Computer Science, USA

² University of Waterloo, Combinatorics & Optimization, Canada

Abstract. A trapdoor Memory-Hard Function is a function that is memory-hard to evaluate for any party who does not have a trapdoor, but is substantially less expensive to evaluate with the trapdoor. Biryukov and Perrin (Asiacrypt 2017) introduced the first candidate trapdoor Memory-Hard Function called DIODON, which modifies a Memory-Hard Function called SCRIPT by replacing a hash chain with repeated squaring modulo a composite number $N = pq$. The trapdoor, which consists of the prime factors p and q , allows one to compute the function with significantly reduced cumulative memory cost (CMC) $O(n \log n \log^2 N)$ where n denotes the running time parameter, e.g., the length of the hash chain or repeated squaring chain. By contrast, the best-known algorithm to compute DIODON without the trapdoor has the CMC $O(n^2 \log N)$. Auerbach et al. (Eurocrypt 2024) provided the first provable lower bound on the CMC of TdSCRIPT — a specific instantiation of DIODON. In particular, in idealized models, they proved that the CMC of TdSCRIPT is $\Omega(\frac{n^2}{\log n} \log N)$ which almost matches the upper bound $O(n^2 \log N)$ but is off by a multiplicative $\log n$ factor. In this work, we show how to tighten the analysis of Auerbach et al. (Eurocrypt 2024) and eliminate the gap. In particular, our results imply that TdSCRIPT has the CMC at least $\Omega(n^2 \log N)$.

Keywords: Trapdoor Memory-Hard Function · TdScript · Cumulative Memory Cost · Generic Group Model · Random Oracle Model

1 Introduction

A Memory-Hard Function (MHF) [Per09] is a cryptographic primitive that requires significant memory resources for evaluation. MHFs are used to design egalitarian Proofs of Work and to help protect low-entropy secrets such as user passwords against brute-force attacks in password hashing. After the initial construction of an MHF called SCRIPT by Percival [Per09], there has been a line of work [FLW14, BDK16, BCS16, ABH17, BHK⁺19, BH22] on the construction of MHFs. Those MHF constructions are all *symmetric*, which means that any party will have the same cost to evaluate the function. In some applications

Biryukov and Perrin [BP17] were the first to consider an *asymmetry* in MHFs by introducing a trapdoor, i.e., a secret piece of information that allows the function to be evaluated much more efficiently by the party who knows the secret information. The MHFs that satisfy this property were first called *asymmetrically Memory-Hard Functions* [BP17] and then later called *trapdoor Memory-Hard Functions (TMHFs)* by Auerbach et al. [AGP24]¹. Biryukov and Perrin proposed the first candidate TMHF called DIODON [BP17] which modifies SCRIPT by replacing a hash chain with repeated squaring modulo

E-mail: jblocki@purdue.edu (Jeremiah Blocki), seunghoon.lee@uwaterloo.ca (Seunghoon Lee)

^aWork done while Seunghoon was at Purdue University.

¹Biryukov and Lombard-Platet [BLP23] also used the word “trapdoor” to illustrate functions with the property we described, but they did not explicitly use the term “trapdoor MHFs”.



an unknown large composite number N . The trapdoor is the number N and it allows one to compute the function with significantly less cost than the one without having the trapdoor (i.e., not knowing the order N).

Potential applications of TMHFs include spam mitigation and compromised credential checking services. For the first application, we consider an e-mail server that wants to combat junk e-mail by requiring senders to provide a Proof of Work (PoW) [DN93]. Hashcash [Bac02] is one of the simplest and most natural PoW schemes, but this PoW scheme is not egalitarian. In particular, a spammer could employ customized hardware to reduce the amortized cost of generating a proof of work by several orders of magnitude in comparison to the costs that an honest sender would incur on a personal computer. For example, Application-Specific Integrated Circuits (ASICs) for computing the SHA-256 hashing algorithm are four to five orders of magnitude more energy-efficient than the CPU on a state-of-the-art personal computer, e.g., see [RD17, NBF⁺16]. An alternative (egalitarian) approach would be to require senders to evaluate an MHF whenever they send a message. While this solution addresses the cost inequity between a spammer (ASIC) and an honest sender (PC), the e-mail server will most likely be overloaded quickly, as the only way to verify that each MHF evaluation is correct would be to repeat each expensive computation. A trapdoor MHF would address this problem. In particular, the e-mail server uses the trapdoor to efficiently verify the sender’s claims while using significantly less resources than the sender.

Compromised credential checking (C3) services [TPY⁺19, LPA⁺19] provide another potentially useful application for TMHFs. C3 services like HaveIBeenPwned [Hun20] and Google Password Checkup [PTB19] allow users to proactively check whether or not their credentials are present in known data breaches. MHFs are used to mitigate the risk of a breach extraction attack where a malicious party tries to use the C3 service to extract credentials from breaches that s/he may not have access to. While MHFs slow down a malicious party by requiring them to perform separate memory-hard computation for each query, the C3 service needs to evaluate the MHF on every breached credential in its breached dataset. This introduces a significant burden for setting up a C3 service, as there are hundreds of billions of leaked credentials to hash. A C3 service could potentially use a TMHF to reduce setup costs while simultaneously providing stronger mitigation against breach extraction attacks. After setup, the TMHF trapdoor could be stored offline or even discarded.

An important cost metric to understand the memory-hardness of (T)MHFs is *cumulative memory cost (CMC)* [AS15] which refers to the total amount of memory required to evaluate the function over a sequence of operations (see Section 2.3 for further details). Biryukov and Perrin [BP17] showed that the best-known algorithm to compute DIODON without the trapdoor has the CMC $O(n^2 \log N)$ where n denotes the running time parameter, i.e., the number of repeated squaring. With the trapdoor, the CMC of DIODON is significantly reduced to $O(n \log n \log^2 N)$.

Auerbach et al. [AGP24] provided the first provable lower bound of the CMC of DIODON. In particular, they considered a specific instantiation of DIODON which mostly resembles SCRIPT, which they named TDSRIPT.

Review: Description of TdScript. TDSRIPT [AGP24] is a concrete instantiation of DIODON [BP17] by setting $M := n$, $L := n$, and $\eta := 1$, where M is a parameter to control the memory-hardness of DIODON and the parameters L and η decide its time complexity, following the same notation from Biryukov and Perrin [BP17]. TDSRIPT is defined over the quadratic residues $\text{QR}_{N'} := \{Q \in \mathbb{Z}_{N'}^* : \exists x \text{ s.t. } x^2 \equiv Q \pmod{N'}\}$ and consists of the following three algorithms. Here, $N' := p'q'$ where p' and q' are a pair of “safe primes”².

² p', q' are called *safe primes* if they are of the same bit length and if $p := (p' - 1)/2$ and $q := (q' - 1)/2$ are also distinct prime with the same bit length $\|p\| = \|q\| = k(\lambda)$ where $k(\lambda) \in \text{poly}(\lambda)$.

(Note: the exponentiations W_{i-1}^2 in $\text{Eval}(\text{pp}, W)$ and $W_0^m, W_0^{m_i}$ in $\text{TDEval}(\text{pp}, \text{td}, W)$ are computed in $\text{QR}_{N'}$.)

- **Setup**(1^λ): the setup algorithm first samples two distinct safe primes p', q' and a hash function $H : \{0, 1\}^* \rightarrow \{0, 1\}^{\omega_H}$. Then it returns public parameters $\text{pp} := (N' := p'q', H, n)$ and the trapdoor $\text{td} := N$ where $N := (p' - 1)(q' - 1)/4$.³
- **Eval**(pp, W): the deterministic evaluation algorithm (without trapdoor) on input $W \in \text{QR}_{N'}$ first computes W^{2^n} by repeated squaring, i.e., it sets $W_0 := W$ and computes $W_i := W_{i-1}^2$ for $i \in [n]$. Now, the algorithm sets $S_0 := H(W_n, 0^{\omega_H})$ and computes $S_i := H(W_{j_i}, S_{i-1})$ for $i \in [n]$, where $j_i := S_{i-1} \bmod n$. Then the algorithm returns $S_n \in \{0, 1\}^{\omega_H}$.
- **TDEval**(pp, td, W): the deterministic trapdoor evaluation algorithm on input $W = W_0$ with the trapdoor $\text{td} = N$ could compute S_n efficiently (without storing all the W_i 's) using the knowledge of $N = (p' - 1)(q' - 1)/4 = |\text{QR}_{N'}|$. In particular, it first computes $m := 2^n \bmod N$ and $W_n := W_0^m$. Then it computes $S_0 := H(W_n, 0^{\omega_H})$. Similar to **Eval**(pp, W), the algorithm then computes $S_i := H(W_{j_i}, S_{i-1})$ for $i \in [n]$, where $j_i := S_{i-1} \bmod n$, but instead of storing all the W_{j_i} 's prior, it computes $m_i := 2^{j_i} \bmod N$ and $W_{j_i} := W_0^{m_i}$ for $i \in [n]$. Finally, the algorithm returns $S_n \in \{0, 1\}^{\omega_H}$.

Remark 1. The input W for **Eval** and **TDEval** does not have to be a quadratic residue but it can be an arbitrary bitstring. To convert W into an element of the quadratic residue group, one could hash it using a separate random oracle then square the result modulo N , i.e., for a random oracle H , we compute $\tilde{W} := H(W)^2 \bmod N$. Then the algorithm **Eval**(pp, W) computes $\tilde{W}^{2^n}$ instead of W^{2^n} by initializing $W_0 := \tilde{W}$ and iteratively setting $W_i = W_{i-1}^2$ for $i \in [n]$. Similarly, **TDEval**(pp, td, W) can also begin by setting $W_0 = \tilde{W}$ and follow with analogous computations.

Review: CMC Lower Bound of TdScript. Auerbach et al. [AGP24] proved that without the trapdoor the CMC of evaluating TdScript is $\Omega\left(\frac{n^2}{\log n} \log N\right)$, which almost matches the upper bound $O(n^2 \log N)$ but off by a multiplicative factor of $\log n$. They proved the CMC lower bound in the Generic Group Model (GGM) [Sho97] and the Random Oracle Model (ROM) [BR93]. In particular, they considered the *Parallel Oracle Model* where a polynomial-time algorithm $\mathcal{A}^{\mathcal{G}, H}$ has access to the generic group oracle $\mathcal{G}$ (which performs group operations) and the random oracle H (see Section 2.2 for the formal description of the Parallel Oracle Model).

A key step in their CMC lower bound was to prove a space-time lower bound for *any* probabilistic polynomial-time (PPT) algorithm in the single-challenge game. Intuitively, in this game an online attacker $\mathcal{A}_{\text{on}}$ is given an initial M bit state str and a uniformly random challenge $j \in [n]$ and is asked to compute W_j as quickly as possible. An M bit state allows us to store $\{W_i\}_{i \in S}$ for a set $S \subseteq [n]$ of size $|S| = \rho = M/\log N'$. Thus, the most natural strategy for the attacker is to store $\{W_{in/\rho}\}_{0 \leq i < \rho}$. Now we can write any challenge j as $j = in/\rho + k$ for some $k < n/\rho$ and then compute W_j within k sequential rounds of repeated squaring starting from $W_{in/\rho}$. On average, it will take the attacker approximately $t \geq (n/\rho - 1)/2$ rounds to respond to a random challenge $j \in [n]$ following this strategy. A natural conjecture is that this is essentially best that the attacker can do, i.e., given space $M = \rho \log N'$ to store at most ρ group elements in $\text{QR}_{N'}$ from our group, it will take at least $t = \Omega(n/\rho)$ rounds to respond to a random challenge j in expectation. However, even in idealized models like the GGM is quite difficult to prove. Auerbach et

³Since p', q' are safe primes, the trapdoor $N = (p' - 1)(q' - 1)/4 = pq$ is also the product of two distinct primes. In this case, it is known that $\text{QR}_{N'} = \langle g \rangle$ is a cyclic group of order N .

al. [AGP24] were able to prove the slightly weaker claim that $t = \Omega(n/(\rho \log n))$ in the GGM, i.e., the natural strategy is asymptotically optimal up to a factor of $\log n$. This lower bound for the single challenge game is sufficient to show that TdSCRIPT has CMC $\Omega\left(\frac{n^2}{\log n} \log N\right)$ — intuitively computing TdSCRIPT is like playing the single-challenge response game n times.

Let g be a generator of the cyclic group $\text{QR}_{N'} = \langle g \rangle$ of order N . In the GGM, each element of the group $\text{QR}_{N'}$ is encoded using a random injective mapping $\Sigma : \mathbb{Z}_N \rightarrow \mathcal{L} := \{0, 1\}^{\omega_{\mathcal{L}}}$, i.e., the element $w = g^w \in \text{QR}_{N'}$ is encoded as $\Sigma(w) \in \{0, 1\}^{\omega_{\mathcal{L}}}$. A GGM algorithm can only interact with the group via generic group oracles $\mathcal{G} = (\text{Add}, \text{Sub})$. Given $\sigma_1, \sigma_2 \in \mathcal{L}$, $\text{Add}(\sigma_1, \sigma_2) := \Sigma(\Sigma^{-1}(\sigma_1) + \Sigma^{-1}(\sigma_2))$ and $\text{Sub}(\sigma_1, \sigma_2) := \Sigma(\Sigma^{-1}(\sigma_1) - \Sigma^{-1}(\sigma_2))$. Observe that if $\sigma = \Sigma(w)$ is the encoding of $W = g^w \in \text{QR}_{N'}$ then $\text{Add}(\sigma, \sigma) = \Sigma(w + w)$ is the encoding of $W^2 = g^{2w} \in \text{QR}_{N'}$.

Auerbach et al. [AGP24] conducted their analysis in the *preprocessing setting*, i.e., we view an algorithm as a pair of deterministic polynomial-time algorithms $\mathcal{A}^{\mathcal{G}, \text{H}} = (\mathcal{A}_{\text{pre}}^{\mathcal{G}, \text{H}}, \mathcal{A}_{\text{on}}^{\mathcal{G}, \text{H}})$ where the preprocessing algorithm $\mathcal{A}_{\text{pre}}^{\mathcal{G}, \text{H}}(\Sigma(1), \Sigma(w))$ takes as input the encoding of g and $W = g^w$ and outputs an M -bit hint str for the online attacker after interacting with the generic group oracles $\mathcal{G} = (\text{Add}, \text{Sub})$. The offline attacker could compute the encoding $\Sigma(w2^i)$ of $W_i = g^{w2^i}$ for each $i \leq n$, but since $M < n\omega_{\mathcal{L}}$ it will not be possible to store each of these values into the hint str , i.e., the hint could store at most $\rho := \lfloor M/\omega_{\mathcal{L}} \rfloor$ encoded group elements. The online attacker $\mathcal{A}_{\text{on}}^{\mathcal{G}, \text{H}}(\text{str}, j)$ is given the M -bit hint and a challenge $j \in [n]$ and attempts to compute $\Sigma(w2^j)$ as quickly as possible. We will let $J \subseteq [n]$ denote the set of challenges $j \in [n]$ such that $\mathcal{A}_{\text{on}}^{\mathcal{G}, \text{H}}(\text{str}, j)$ successfully outputs $\Sigma(w2^j)$ after at most t rounds.

The reduction assigns each label queried or received by $\mathcal{A}_{\text{on}}^{\mathcal{G}, \text{H}}$ a unique algebraic expression over $\mathbb{Z}_N$, using indeterminates, and stores these associations in a table to keep track of consistency. For instance, when $\mathcal{A}_{\text{on}}^{\mathcal{G}, \text{H}}$ submits a query like $\text{Add}(\sigma_1, \sigma_2)$, the system checks if σ_1 and σ_2 already have representations; if not, it assigns them new indeterminates. It then computes the representation for the resulting label as the sum of the representations for σ_1 and σ_2 . For a concrete example, suppose that σ_1 had a representation with an indeterminate x_1 (i.e., we assign x_1 as an unknown value such that $\sigma_1 = \Sigma(x_1)$) and σ_2 did not have one in the table. If $\mathcal{A}_{\text{on}}^{\mathcal{G}, \text{H}}$ submits a query $\text{Add}(\sigma_1, \sigma_2)$, then the reduction adds a new indeterminate x_2 which represents σ_2 , and output $\text{Add}(\sigma_1, \sigma_2) = \sigma_3$ where the representation for σ_3 is $x_1 + x_2$, i.e., $\sigma_3 = \Sigma(x_1 + x_2)$. If the adversary succeeded to query the label $\Sigma(w2^j)$ after t rounds and the table contains m indeterminates, we get the algebraic equation

$$a_{j,1}x_1 + \dots + a_{j,m}x_m = w2^j \pmod{N},$$

where $a_{j,1}, \dots, a_{j,m}$ are integers.

Auerbach et al. [AGP24] demonstrated that for every $i \in [m]$, the absolute value of each coefficient $|a_{j,i}|$ is bounded by 2^t . This bound is reached when operations are repeatedly applied to the same indeterminate over the entire t rounds. For instance, with x_1 representing σ_1 , the query $\sigma_2 \leftarrow \text{Add}(\sigma_1, \sigma_1)$ produces $x_1 + x_1 = 2x_1$, and $\sigma_3 \leftarrow \text{Add}(\sigma_2, \sigma_2)$ results in $2x_1 + 2x_1 = 4x_1$. Continuing this process, the representation for $\sigma_{t+1} \leftarrow \text{Add}(\sigma_t, \sigma_t)$ after the final round becomes $2^t x_1$. Since no additional indeterminates are introduced beyond x_1 , this process yields the maximum value for any $a_{j,i}$.

The reduction considers a subset $J = \{j_1, \dots, j_\ell\} \subseteq [0, n]$ of ℓ challenges that can be answered within $\leq t$ rounds, and therefore, the single-challenge game reduces to solving the system of linear equations $A\mathbf{x} = \mathbf{b}$ where $A = (a_{j,i})_{j \in [\ell], i \in [m]} \in \mathbb{Z}^{\ell \times m}$ with $|a_{j,i}| \leq 2^t$ for all $j \in [\ell], i \in [m]$ and vector $\mathbf{b} = (2^{j_1}w, \dots, 2^{j_\ell}w)^\top$ with pairwise distinct $j_1, \dots, j_\ell \in \mathbb{N}$. Here, we remark that the system of equations is defined over $\mathbb{Z}$ instead of $\mathbb{Z}_N$ since the attacker does not have the trapdoor $\text{td} = N$.

Then Auerbach et al. [AGP24, Lemma 2] gave a crucial Lemma that lower bounds the rank of A (denoted by $\text{rank}(A)$) from the condition $|a_{j,i}| \leq 2^t$ for all $j \in [\ell]$ and $i \in [m]$.

Intuitively, $\text{rank}(A)$ denotes the number of indeterminates that can be extracted from the system of equations. In particular, they proved that $\text{rank}(A) \geq \ell/(3 \max\{t, \log \ell\})$. A separate incompressibility argument [DTT10, AGP24] implies that we can also upper bound $\text{rank}(A)$ as $\text{rank}(A) \leq c\rho$ for some constant $c > 0$. Taken together, they argued that the running time for the single-challenge game (denoted by `TimeSC`; see Definition 5 for the formal definition) is lower bounded by $n/(6\rho \log(n/2))$ with probability $> 1/2$ where $\rho = \lfloor M/\omega_{\mathcal{L}} \rfloor = \Theta(M/\log N)$ is the size of the hint `str` normalized by the number of bits to encode a group element. From this point, Auerbach et al. [AGP24] are able to establish the CMC lower bound $\Omega\left(\frac{n^2}{\log n} \log N\right)$ for evaluating `TdSCRYPT` by adapting analysis of the `SCRYPT` Memory-Hard Function [ACP⁺17].

On the Non-Tightness of the Lower Bound. As mentioned before, the CMC lower bound $\Omega\left(\frac{n^2}{\log n} \log N\right)$ [AGP24] is not tight and off from the upper bound $O(n^2 \log N)$ by a $\log n$ factor. This non-tightness occurred due to the sub-optimal lower bound of $\text{rank}(A) \geq \ell/(3 \max\{t, \log \ell\})$ where the bound contains $\log \ell$ term, which leads to having $\log n$ term in the denominator of the CMC lower bound. Hence, the question is whether we can eliminate the $\log \ell$ term from [AGP24, Lemma 2], i.e., if we can prove $\text{rank}(A) \geq \ell/(ct)$ for some constant $c > 0$.

A key idea to prove [AGP24, Lemma 2] was to prove the following combinatorial claim: for a subset $R \subset \mathbb{Z}^m$ with $|R| = \ell$ and every $\mathbf{r} \in R$ with $\|\mathbf{r}\|_{\infty} \leq 2^t$, if $m < \ell/(3 \max\{t, \log \ell\})$ then there exist two distinct and disjoint subsets $R_1, R_2 \subseteq R$ such that their sums are equal, i.e., $\sum_{\mathbf{r}_1 \in R_1} \mathbf{r}_1 = \sum_{\mathbf{r}_2 \in R_2} \mathbf{r}_2$ (see [AGP24, Claim 4]). Here, they only use the infinity norm ($\|\mathbf{r}\|_{\infty} \leq 2^t$) to bound on rows but we observe that a stronger L_1 -norm bound (i.e., $\|\mathbf{r}\|_1 \leq 2^t$) can indeed be shown (as noted in [AGP24, Appendix B]). It was an open question whether we can utilize this stronger L_1 -norm bound to eliminate the $\log \ell$ term from [AGP24, Claim 4] which consequently eliminates the $\log \ell$ term from [AGP24, Lemma 2] as well.

1.1 Our Contributions

In this work, we resolve the problem raised above and prove a tighter rank bound for the system of linear equations $A\mathbf{x} = \mathbf{b}$. In particular, we prove an improved lower bound $\text{rank}(A) \geq \ell/(2et)$ where e is Euler's number (see Lemma 2) which does not include $\log \ell$ term in the denominator compared to [AGP24, Lemma 2].

Lemma 2. *Let $\ell, t, m, w \in \mathbb{N}$. For any matrix $A = (a_{i,j})_{i \in [\ell], j \in [m]} \in \mathbb{Z}^{\ell \times m}$ with $\sum_{j=1}^m |a_{i,j}| \leq 2^t$ for all $i \in [\ell]$ and vector $\mathbf{b} = (2^{j_1 w}, \dots, 2^{j_{\ell} w})^{\top}$ with pairwise distinct $j_1, \dots, j_{\ell} \in \mathbb{N}$, if the system $A\mathbf{x} = \mathbf{b}$ has a solution $\mathbf{x} \in \mathbb{Z}^m$, then*

$$\text{rank}(A) \geq \frac{\ell}{2et}.$$

Consequently, we prove a *tight* CMC lower bound $\Omega(n^2 \log N)$ for evaluating `TdSCRYPT` as stated in Theorem 1.

Theorem 1. *Let $n \in \text{poly}(\lambda)$ with $n \geq 8$ and let $\mathcal{A}$ be a deterministic parallel oracle machine that evaluates `TdSCRYPT` correctly with probability $\chi(\lambda)$ over the choice of the parameters and input W . Then, assuming that the factoring is hard, in the GGM and ROM with probability at least $\chi(\lambda) - \epsilon(\lambda)$,*

$$\text{cmc}(\mathcal{A}^{\Sigma, \mathbf{H}}(\Sigma(1), \Sigma(w))) \in \Omega(n^2 \log N),$$

where $\epsilon(\lambda) \in \text{negl}(\lambda)$, and the probability is taken over $(N, \Sigma, \mathbf{H}, w) \leftarrow \text{params}(\lambda)$.

A key ingredient of our result is proving a stronger version of [AGP24, Claim 4] which uses a stronger L_1 -norm bound instead of the infinity-norm bound.

Claim 2. *For $t, m, \ell \in \mathbb{N}$ with $t \geq 2$ and $\ell \geq 4$, let $R \subset \mathbb{Z}^m$ with $|R| = \ell$ and every $\mathbf{r} \in R$ satisfying $\|\mathbf{r}\|_1 \leq 2^t$. If $m < \ell/(2et)$, two subsets $R_1, R_2 \subseteq R$ exist such that at least one subset is non-empty ($R_1 \cup R_2 \neq \emptyset$), they are disjoint ($R_1 \cap R_2 = \emptyset$), and their sums are equal ($\sum_{\mathbf{r}_1 \in R_1} \mathbf{r}_1 = \sum_{\mathbf{r}_2 \in R_2} \mathbf{r}_2$).*

We stress that even though the possibility of using a stronger L_1 -norm bound for $\mathbf{r}$ was already mentioned in the original paper [AGP24], proving Claim 2 is not a simple extension of prior work and requires nontrivial proof techniques. See Section 3 for the details.

2 Preliminaries

Throughout the paper, e denotes Euler’s number. Unless otherwise noted, all log’s have base 2, i.e., $\log x := \log_2 x$. For a positive integer n , $[n] := \{1, \dots, n\}$ and $[0, n) := \{0, \dots, n-1\}$. We follow standard set notations: $\mathbb{N}$ denotes the set of whole numbers, $\mathbb{Z}$ denotes the set of integers, $\mathbb{Z}_N$ denotes the set of integers modulo N , and $\mathbb{Z}_N^* := \{x \in \mathbb{Z}_N : \gcd(x, N) = 1\}$ denotes the set of integers in $\mathbb{Z}_N$ that is coprime to N . The notation $\leftarrow \$$ denotes a uniformly random sampling, e.g., we say $x \leftarrow \$ \mathbb{Z}_N$ when x is sampled uniformly at random from $\mathbb{Z}_N$.

2.1 Generic Group Model

The *Generic Group Model (GGM)* is an idealized model to analyze the security of cryptographic schemes that rely on group operations. In the GGM, all group operations are performed via queries to a generic group oracle, without direct access to the concrete representation of the group elements. As a result, group operations become “generic” in the sense that one cannot exploit any specific algebraic structure of the group. The use of generic groups dates back to Nechaev [Nec94], and the GGM was later formally defined by Shoup [Sho97] and Maurer [Mau05].

While there exist various types of GGMs [Sho97, SJ00, Mau05, KMP16], we adopt Shoup’s model [Sho97] in this work, which encodes group elements using random injective labels. This choice aligns with the prior analysis of TdScript by Auerbach et al. [AGP24], which was also conducted in Shoup’s GGM. Unlike Shoup’s model, other GGMs do not use random injective labeling for group elements; Maurer’s model [Mau05] and Kiltz et al.’s model [KMP16] uses pointers instead of random labels, and Schnorr and Jakobsson’s model [SJ00] only provides indirect access to group elements. In particular, if f_i is the output of the i^{th} generic group query, then the model only informs whether (1) f_i is a new group element, and (2) f_i collided with a prior query.

Some GGMs such as Maurer’s model [Mau05] and Schnorr and Jakobsson’s model [SJ00] fail to capture any attacks that exploit the ability to hash group elements, including preprocessing attacks of Corrigan-Gibbs and Kogan [CK18] — see the discussion of [BL22]. Blocki and Lee [BL22] further demonstrated a generic transformation of any attack in Kiltz et al.’s GGM [KMP16] into the Schnorr and Jakobsson’s model, implying that Kiltz et al.’s GGM is also not suitable to analyzing such attacks. Additionally, Zhandry [Zha22] showed that Maurer’s model fails to capture fundamental cryptographic techniques such as the Merkle-Damgård transform, Feistel network, and authenticated encryption, as these techniques require a certain way to interpret group elements as strings. Given these limitations, we adopt Shoup’s GGM [Sho97] throughout this paper.

Group operations in Shoup’s GGM work as follows. Consider a cyclic group $G = \langle g \rangle$ of order N and a random injective labeling map $\Sigma : \mathbb{Z}_N \rightarrow \mathcal{L} := \{0, 1\}^{\omega_{\mathcal{L}}}$. Here, $\omega_{\mathcal{L}} \geq \lceil \log N \rceil$.

to ensure that the map Σ is injective. Generic group oracles $\mathcal{G} = (\text{Add}, \text{Sub})$ allow computations $\text{Add}(\mathbf{a}, \mathbf{b}) := \Sigma(\Sigma^{-1}(\mathbf{a}) + \Sigma^{-1}(\mathbf{b}))$ and $\text{Sub}(\mathbf{a}, \mathbf{b}) := \Sigma(\Sigma^{-1}(\mathbf{a}) - \Sigma^{-1}(\mathbf{b}))$ for $\mathbf{a}, \mathbf{b} \in \mathcal{L}$ (if either $\mathbf{a} \notin \Sigma(\mathbb{Z}_N)$ or $\mathbf{b} \notin \Sigma(\mathbb{Z}_N)$, it returns $\perp$). Note that one can efficiently compute the inverse of $\mathbf{a}$ using generic group oracles: (1) obtain $\eta \leftarrow \text{Sub}(\mathbf{a}, \mathbf{a}) = \Sigma(0)$, then compute (2) $\text{Inv}(\mathbf{a}) \leftarrow \text{Sub}(\eta, \mathbf{a})$. We remark that in this paper, we particularly consider the GGM for the quadratic residue group $\text{QR}_{N'}$ with $N' = p'q'$ where p', q' are a pair of safe primes. Then it is known that $\text{QR}_{N'} = \langle g \rangle$ becomes a cyclic group of order $N = (p' - 1)(q' - 1)/4$ (see footnote 2 (Page 2) and footnote 3 (Page 3)).

2.2 Parallel Oracle Model

Auerbach et al. defined a machine model called the *Parallel Oracle Model* [AGP24] to properly define a memory complexity metric. Intuitively, the Parallel Oracle Model works the same as the Parallel Random Oracle Model [AS15, ACP⁺17] with additional access to the generic group oracle queries. In the Parallel Oracle Model, a polynomial-time algorithm $\mathcal{A}^{\mathcal{G}, \text{H}}$ has access to the random oracle $\text{H} : \{0, 1\}^* \rightarrow \{0, 1\}^{\omega_{\text{H}}}$ (where $\omega_{\text{H}} \in \Theta(\log N)$) as well as the generic group oracle $\mathcal{G} = (\text{Add}, \text{Sub})$. As mentioned in [AGP24], we will omit the superscript and simply write $\mathcal{A}$ if the superscript is clear from context. As the name suggests, the Parallel Oracle Model allows the algorithm to make queries in parallel. In particular, the algorithm $\mathcal{A}^{\mathcal{G}, \text{H}}$ runs in rounds and it can make a batch of queries in each round. In round i , $\mathcal{A}^{\mathcal{G}, \text{H}}$ receives the response from the previous round and produces an output state $\overline{\mathbf{st}}_i := (\tau_i, \mathbf{qrs}_i^{\mathcal{G}}, \mathbf{qrs}_i^{\text{H}})$ where τ_i is a bitstring and $\mathbf{qrs}_i^{\mathcal{G}}$ (resp. $\mathbf{qrs}_i^{\text{H}}$) is the vector containing queries to $\mathcal{G}$ (resp. H). In the next round (round $i + 1$), $\mathcal{A}^{\mathcal{G}, \text{H}}$ receives the input state $\mathbf{st}_i := (\tau_i, \mathbf{res}_i^{\mathcal{G}}, \mathbf{res}_i^{\text{H}})$ where $\mathbf{res}_i^{\mathcal{G}}$ (resp. $\mathbf{res}_i^{\text{H}}$) is the vector containing the response from queries $\mathbf{qrs}_i^{\mathcal{G}}$ (resp. $\mathbf{qrs}_i^{\text{H}}$).

2.3 Cumulative Memory Cost

The *Cumulative Memory Cost (CMC)* [AS15] is a key metric used to evaluate the memory hardness of MHFs. The CMC measures the total memory usage of an algorithm throughout its execution. Specifically, consider an algorithm $\mathcal{A}^{\mathcal{G}, \text{H}}$ with input x as introduced in Section 2.2, which produces a sequence of input states $\mathbf{st}_i = (\tau_i, \mathbf{res}_i^{\mathcal{G}}, \mathbf{res}_i^{\text{H}})$ during its execution. If $\mathcal{A}^{\mathcal{G}, \text{H}}$ runs for T rounds, its CMC is defined as

$$\text{cmc}(\mathcal{A}^{\mathcal{G}, \text{H}}(x)) := \sum_{i=1}^T \|\mathbf{st}_i\|,$$

where $\|\mathbf{st}_i\| := \|\tau_i\| + |\mathbf{res}_i^{\mathcal{G}}| \omega_{\mathcal{L}} + |\mathbf{res}_i^{\text{H}}| \omega_{\text{H}}$ represents the bit length of the i^{th} input state. By convention, an algorithm submits at least one query per round, ensuring that $\|\mathbf{st}_i\| \geq \min\{\omega_{\mathcal{L}}, \omega_{\text{H}}\} \in \Theta(\log N)$.

We remark that the CMC provides a more reliable measure of memory hardness than the space-time cost, as it better captures resistance to parallelism and amortization. Alwen and Serbinenko [AS15] observed that parallelism can significantly reduce the space-time cost of an MHF. Consider an evaluation algorithm $\mathcal{A}$ that consumes a large amount of memory only during an initial phase, after which it requires only low memory. By running multiple copies of $\mathcal{A}$ in parallel (denoted by $\mathcal{A}^{\otimes n}$), one can *amortize* the space-time cost, keeping it nearly the same as that of a single execution⁴. However, the CMC ensures that $\text{cmc}(\mathcal{A}^{\otimes n}) = n \cdot \text{cmc}(\mathcal{A})$, implying that the CMC remains robust against such optimizations.

⁴In particular, let $d \ll T$ be the number of steps where $\mathcal{A}$ consumes a lot of memory during an initial phase and $s \ll \max_t S(t)$ be the (low) memory usage for the rest of the execution of $\mathcal{A}$. One can amortize the space-time cost using parallelism, i.e., start evaluating $\mathcal{A}$ right after the previous execution finishes its initial phase (where it consumes high memory). Then the total space-time cost to run $\mathcal{A}^{\otimes n}$ becomes $(T + (n - 1)d)(\max_t S(t) + (n - 1)s) \approx T \cdot \max_t S(t)$, which is the space-time cost of running $\mathcal{A}$ only once.

2.4 Hardness Assumptions

As introduced in the prior work [AGP24], we give two hardness assumptions. They first specified the set of parameters which determine an execution of TdSCRIPT. For completeness, we cite those definitions quoted verbatim.

Definition 1 ([AGP24, Definition 1]). Let GenSP be an algorithm that samples two distinct safe primes of the same bit length uniformly at random. More precisely, define $(p', q') \leftarrow \text{GenSP}(\lambda)$ and let $p := (p' - 1)/2$ and $q := (q' - 1)/2$. It holds that p', q', p , and q are prime such that $p \neq q$ and $\|p\| = \|q\| = k(\lambda)$ with $k(\lambda) \in \text{poly}(\lambda)$.

Definition 2 ([AGP24, Definition 2], Factoring N'). The game $\text{Fac}'_{\text{GenSP}, \mathcal{A}}(\lambda)$ is defined with respect to GenSP and a *probabilistic polynomial-time (PPT)* adversary $\mathcal{A}$. On input of a security parameter λ , it runs $(p', q') \leftarrow \text{GenSP}(\lambda)$, sets $N' := p'q'$, and invokes the adversary, yielding $(p^*, q^*) \leftarrow \mathcal{A}(N')$. The game returns 1 if $\{p^*, q^*\} = \{p', q'\}$ and 0 otherwise.

We say that *factoring N' is hard* if, for every PPT algorithm $\mathcal{A}$, the advantage is negligible, i.e.,

$$\text{Adv}_{\text{GenSP}, \mathcal{A}}^{\text{Fac}'}(\lambda) := \Pr[1 \leftarrow \text{Fac}'_{\text{GenSP}, \mathcal{A}}(\lambda)] \in \text{negl}(\lambda),$$

where the probability is taken over the randomness of GenSP and $\mathcal{A}$.

Definition 3 ([AGP24, Definition 3], Factoring N). The game $\text{Fac}_{\text{GenSP}, \mathcal{A}}(\lambda)$ is defined similarly to $\text{Fac}'_{\text{GenSP}, \mathcal{A}}(\lambda)$ but with N' replaced by N . So the game invokes $(p^*, q^*) \leftarrow \mathcal{A}(N)$ and returns 1 if and only if $\{p^*, q^*\} = \{p', q'\}$. We say that *factoring N is hard* if, for every PPT algorithm $\mathcal{A}$,

$$\text{Adv}_{\text{GenSP}, \mathcal{A}}^{\text{Fac}}(\lambda) := \Pr[1 \leftarrow \text{Fac}_{\text{GenSP}, \mathcal{A}}(\lambda)] \in \text{negl}(\lambda),$$

where the probability is taken over the randomness of GenSP and $\mathcal{A}$.

For simplicity, we say that *factoring is hard* with respect to GenSP if both factoring N and N' are hard with respect to GenSP .

3 Improved Lower Bound of $\text{rank}(A)$

Given a system of linear equations $A\mathbf{x} = \mathbf{b}$, Auerbach et al. [AGP24] showed the lower bound of $\text{rank}(A)$ if there is an upper bound for the entries of A , i.e., $A = (a_{i,j})_{i \in [\ell], j \in [m]}$ with $|a_{i,j}| \leq 2^t$ for all $i \in [\ell]$ and $j \in [m]$, which is stated in Lemma 1 for completeness.

Lemma 1 ([AGP24, Lemma 2]). Let $\ell, t \in \mathbb{N}$ with $\ell \geq 4$ and $m, w \in \mathbb{N}$. For any matrix $A = (a_{i,j})_{i \in [\ell], j \in [m]} \in \mathbb{Z}^{\ell \times m}$ with $|a_{i,j}| \leq 2^t$ for all $i \in [\ell]$ and vector $\mathbf{b} = (2^{j_1}w, \dots, 2^{j_\ell}w)^\top$ with pairwise distinct $j_1, \dots, j_\ell \in \mathbb{N}$, if the system $A\mathbf{x} = \mathbf{b}$ has a solution $\mathbf{x} \in \mathbb{Z}^m$, then

$$\text{rank}(A) \geq \frac{\ell}{3 \max\{t, \log \ell\}}.$$

A key idea to prove Lemma 1 was to prove the following combinatorial claim.

Claim 1 ([AGP24, Claim 4]). For $t, m, \ell \in \mathbb{N}$ with $\ell \geq 4$, let $R \subset \mathbb{Z}^m$ with $|R| = \ell$ and every $\mathbf{r} \in R$ satisfying $\|\mathbf{r}\|_\infty \leq 2^t$. If $m < \ell/(3 \max\{t, \log \ell\})$, two subsets $R_1, R_2 \subseteq R$ exist such that at least one subset is non-empty ($R_1 \cup R_2 \neq \emptyset$), they are disjoint ($R_1 \cap R_2 = \emptyset$), and their sums are equal ($\sum_{\mathbf{r}_1 \in R_1} \mathbf{r}_1 = \sum_{\mathbf{r}_2 \in R_2} \mathbf{r}_2$).

In this work, we improve the condition on m in Claim 1 which is stated in Claim 2, which leads to Lemma 2. While Lemma 2 has a stronger precondition than Lemma 1, we note that stronger precondition does hold, i.e., Auerbach et al. [AGP24] already observed in [AGP24, Lemma 6] that the infinity norm can be improved to the 1-norm.

Lemma 2. *Let $\ell, t, m, w \in \mathbb{N}$. For any matrix $A = (a_{i,j})_{i \in [\ell], j \in [m]} \in \mathbb{Z}^{\ell \times m}$ with $\sum_{j=1}^m |a_{i,j}| \leq 2^t$ for all $i \in [\ell]$ and vector $\mathbf{b} = (2^{j_1}w, \dots, 2^{j_\ell}w)^\top$ with pairwise distinct $j_1, \dots, j_\ell \in \mathbb{N}$, if the system $A\mathbf{x} = \mathbf{b}$ has a solution $\mathbf{x} \in \mathbb{Z}^m$, then*

$$\text{rank}(A) \geq \frac{\ell}{2et}.$$

The proof of [Lemma 2](#) follows the exact same steps as the proof of [Lemma 1](#) (see [\[AGP24, Lemma 2\]](#)) except that we use [Claim 2](#) instead of [Claim 1](#) so that “ $3 \max\{t, \log \ell\}$ ” part is updated to “ $2et$ ”. Hence, it is sufficient to prove [Claim 2](#) to prove [Lemma 2](#). For completeness, we defer the proof of [Lemma 2](#) to the end of the section.

Claim 2. *For $t, m, \ell \in \mathbb{N}$ with $t \geq 2$ and $\ell \geq 4$, let $R \subset \mathbb{Z}^m$ with $|R| = \ell$ and every $\mathbf{r} \in R$ satisfying $\|\mathbf{r}\|_1 \leq 2^t$. If $m < \ell/(2et)$, two subsets $R_1, R_2 \subseteq R$ exist such that at least one subset is non-empty ($R_1 \cup R_2 \neq \emptyset$), they are disjoint ($R_1 \cap R_2 = \emptyset$), and their sums are equal ($\sum_{\mathbf{r}_1 \in R_1} \mathbf{r}_1 = \sum_{\mathbf{r}_2 \in R_2} \mathbf{r}_2$).*

Proof. For any subset’s sum $\sum_{\mathbf{r}_1 \in R_1} \mathbf{r}_1 =: \mathbf{s}_1$, we have $\|\mathbf{s}_1\|_1 \leq \sum_{\mathbf{r}_1 \in R_1} \|\mathbf{r}_1\|_1 \leq 2^t \ell$ since $\|\mathbf{r}_1\|_1 \leq 2^t$ for all $\mathbf{r}_1 \in R_1 \subseteq R$ and there are at most ℓ elements in R_1 . Next, we know that there are 2^ℓ distinct (not necessarily disjoint) subsets.

Suppose that all subset’s sums are distinct. Then by pigeonhole principle, it must hold that

$$2^m \binom{m + 2^t \ell}{m} \geq 2^\ell, \quad (1)$$

since there are at most $2^m \binom{m + 2^t \ell}{m}$ possible $\mathbf{s}_1$ ’s that satisfies $\|\mathbf{s}_1\|_1 \leq 2^t \ell$ by [Claim 3](#). Then by [Claim 4](#), we observe that [Equation \(1\)](#) implies $m \geq \ell/(2et)$, which contradicts the condition $m < \ell/(2et)$. This contradiction occurred due to the assumption that all subset’s sums are distinct. Hence, there exist two distinct, but not necessarily disjoint subsets R'_1 and R'_2 with the equal subset’s sum. Applying the same trick from [\[AGP24, Claim 4\]](#), we can easily get disjoint sets R_1 and R_2 (by defining $R_i := R'_i \setminus (R'_1 \cap R'_2)$ for $i \in [2]$). This completes the proof. $\square$

Claim 3. *For $t, m, \ell \in \mathbb{N}$. Then there are at most $2^m \binom{m + 2^t \ell}{m}$ possible $\mathbf{s} \in \mathbb{Z}^m$ that satisfies $\|\mathbf{s}\|_1 \leq 2^t \ell$.*

Proof. If we restrict our domain to $\mathbb{N}^m$, we observe that any dimension- m vector $\mathbf{s} = (s_1, \dots, s_m) \in \mathbb{N}^m$ that satisfies $\|\mathbf{s}\|_1 \leq 2^t \ell$ can be encoded by a binary string of length $2^t \ell + m$ with Hamming weight exactly m , and therefore the number of such vectors is $\binom{m + 2^t \ell}{m}$. The encoding works as follows: initialize the string $x = \emptyset$ and define $s_{m+1} = 2^t \ell - \|\mathbf{s}\|_1 \geq 0$. Foreach $i = 1$ to m update $x \leftarrow x \circ 0^{s_i} \circ 1$. After the loop set $x = x \circ 0^{s_{m+1}}$ to ensure that the final string has length $m + 2^t \ell$. Given a binary string with Hamming weight m , we can decode the string x by setting $s_1 = (\text{the number of leading 0’s})$ and, more generally, $s_i = (\text{the number of 0’s between the } (i-1)^{\text{th}} \text{ instance of 1 and the } i^{\text{th}} \text{ instance of 1})$ for $i \in [2, m]$.

This can also be interpreted as an instance of the “stars and bars” problem by introducing an additional variable $c \geq 0$ such that $s_1 + \dots + s_m + c = 2^t \ell$. The number of possible tuples $(s_1, \dots, s_m, c)$ corresponds to the number of ways to arrange $2^t \ell$ stars and m bars, which is given by $\binom{m + 2^t \ell}{m}$.

Since we allow for positive/negative entries, the number of $\mathbf{s} \in \mathbb{Z}^m$ is at most $2^m \binom{m + 2^t \ell}{m}$ since we multiply by 2^m to account for all possible sign vectors. This completes the proof. $\square$

Below we give [Claim 4](#) and [Claim 5](#), where [Claim 4](#) is used in the proof of [Claim 2](#) above and [Claim 5](#) is used to prove [Claim 4](#).

Claim 4. If $m < \frac{\ell}{2et}$ then for $t \geq 2$, $2^m \binom{m+2^t\ell}{m} < 2^\ell$.

Proof. We first recall that for any $n \in \mathbb{N}$ and $k \in [n]$ the following holds:

$$\left(\frac{n}{k}\right)^k \leq \binom{n}{k} \leq \left(\frac{en}{k}\right)^k. \quad (2)$$

By [Claim 5](#), the function $f(m) = m(\log e + t + 2 + \log \ell - \log m)$ is continuous and increasing over $m < \frac{\ell}{2et}$. Hence, if $m < \frac{\ell}{2et}$, we have

$$\begin{aligned} m(\log e + t + 2 + \log \ell - \log m) &= f(m) < f\left(\frac{\ell}{2et}\right) \\ &= \frac{\ell}{2et} (2 \log e + t + \log t + 3) \\ &< \ell, \end{aligned} \quad (3)$$

for $t \geq 2$. Hence,

$$\begin{aligned} 2^m \binom{m+2^t\ell}{m} &\leq 2^m \left(\frac{e(m+2^t\ell)}{m}\right)^m && \triangleleft \text{by Equation (2)} \\ &< 2^m \left(\frac{e2^{t+1}\ell}{m}\right)^m && \triangleleft \text{since } m < \frac{\ell}{2et} < 2^t\ell \\ &= 2^{m(\log e + t + 2 + \log \ell - \log m)} \\ &< 2^\ell, && \triangleleft \text{by Equation (3)} \end{aligned}$$

which completes the proof. $\square$

Claim 5. Let $t, \ell \in \mathbb{N}$ be constants. Then the function $f : \mathbb{R} \rightarrow \mathbb{R}$ defined by

$$f(x) := x(\log e + t + 2 + \log \ell - \log x)$$

is continuous and increasing over $x < \frac{\ell}{2et}$.

Proof. It is easy to see that f is continuous and differentiable at every point in $\mathbb{R}$. Now, for $x < \frac{\ell}{2et}$, we observe

$$\begin{aligned} f'(x) &= \log e + t + 2 + \log \ell - \frac{1}{\ln 2} - \log x \\ &> \log e + t + 2 + \log \ell - \frac{1}{\ln 2} - \log\left(\frac{\ell}{2et}\right) \\ &= 2 \log e + t + \log t + 3 - \frac{1}{\ln 2} \\ &> 2 \log e + t + \log t > 0, \end{aligned}$$

which completes the proof. $\square$

Proof of [Lemma 2](#). Suppose for contradiction that there exist a matrix A as well as a vector $\mathbf{b}$ satisfying the constraints in the statement of [Lemma 2](#) and that the system $A\mathbf{x} = \mathbf{b}$ has a solution, but

$$\text{rank}(A) < \frac{\ell}{2et}.$$

We will show that this implies the existence of a vector $\mathbf{v} \in \{-1, 0, 1\}^\ell$ with $\mathbf{v} \neq \mathbf{0}$ and $\mathbf{v}A = \mathbf{0}$.

For finding such a $\mathbf{v}$, we may assume that $\text{rank}(A) = m$ (i.e., A is full-rank). To see this, assume that $A \in \mathbb{Z}^{\ell \times m}$ is not full rank. We will see that it is sufficient to consider $A' \in \mathbb{Z}^{\ell \times \text{rank}(A)}$, the submatrix containing a linearly independent subset of A 's columns that is of maximum size, i.e., $\text{rank}(A)$.

Let the columns of A be $\mathbf{c}_1, \dots, \mathbf{c}_m$ and, without loss of generality, assume that $\mathbf{c}_1, \dots, \mathbf{c}_{\text{rank}(A)}$ are linearly independent, so $A' = (\mathbf{c}_1, \dots, \mathbf{c}_{\text{rank}(A)})$. By definition of linear dependence, for any column j with $\text{rank}(A) < j \leq m$, there exist coefficients α_i (with at least one $\alpha_i \neq 0$) such that

$$\alpha_1 \mathbf{c}_1 + \dots + \alpha_{\text{rank}(A)} \mathbf{c}_{\text{rank}(A)} + \alpha_j \mathbf{c}_j = \mathbf{0}.$$

Multiplying by $\mathbf{v}$, we get

$$\alpha_1 \mathbf{v} \mathbf{c}_1 + \dots + \alpha_{\text{rank}(A)} \mathbf{v} \mathbf{c}_{\text{rank}(A)} + \alpha_j \mathbf{v} \mathbf{c}_j = \mathbf{v} \mathbf{0},$$

which simplifies to $\alpha_j \mathbf{v} \mathbf{c}_j = \mathbf{0}$ since $\mathbf{v} A' = \mathbf{0}$ by assumption. Note that $\alpha_j \neq 0$ as otherwise some α_i where $1 \leq i \leq \text{rank}(A)$ would need to be non-zero which would imply that $\mathbf{c}_1, \dots, \mathbf{c}_{\text{rank}(A)}$ are *not* linearly independent. Dividing by α_j , it follows that $\mathbf{v} \mathbf{c}_j = \mathbf{0}$ for all $\text{rank}(A) < j \leq m$ which implies $\mathbf{v} A = \mathbf{0}$. It follows that we can restrict our attention to the full-rank case.

Now we use [Claim 2](#) to construct $\mathbf{v}$. Whenever A contains two identical rows, it is straightforward to define an appropriate $\mathbf{v}$. Thus, we may assume that the rows of A are distinct and form a set $R \subset \mathbb{Z}^m$ of size $|R| = \ell$. Combining this with the assumption $m = \text{rank}(A) < \ell/(2\epsilon)$, [Claim 2](#) applies, so there exist two disjoint subsets R_1 and R_2 with equal sum with one of them being non-empty. Hence, we initialize $\mathbf{v} := \mathbf{0}$, and set $v_i := 1$ if $\mathbf{r}_i \in R_1$ and $v_j := -1$ if $\mathbf{r}_j \in R_2$. This is unambiguous as the subsets are disjoint. Further, notice that $\mathbf{v} \neq \mathbf{0}$ since one subset is non-empty, and, since the subsets' sums are equal, $\mathbf{v} A = \mathbf{0}$ as required.

Last, we apply [Lemma 3](#) to $A\mathbf{x} = \mathbf{b}$ and $\mathbf{v}$. By definition, $\mathbf{b} \in \mathbb{Z}^\ell$ contains no element $b_i = 0$, all b_i are pairwise distinct and exponentially far apart. Thus, $\mathbf{v} \mathbf{b} = \beta \in \mathbb{Z}$ with $\beta \neq 0$ since $\mathbf{v} \in \{-1, 0, 1\}^\ell$. Setting $\mathbf{v}' := \frac{1}{|\beta|+1} \mathbf{v}$, we can conclude that the system $A\mathbf{x} = \mathbf{b}$ does not have a solution by van der Waerden ([Lemma 3](#)). This is a contradiction, completing the proof. $\square$

Lemma 3 ([AGP24, Lemma 3], van der Waerden). *Consider the Diophantine system of linear equations $A\mathbf{x} = \mathbf{b}$ with $A \in \mathbb{Z}^{\ell \times m}$ and $\mathbf{b} \in \mathbb{Z}^\ell$. The system has a solution $\mathbf{x} \in \mathbb{Z}^m$ if and only if, for every rational vector $\mathbf{v} \in \mathbb{Q}^\ell$ such that $\mathbf{v} A \in \mathbb{Z}^m$, it also holds that $\mathbf{v} \mathbf{b} \in \mathbb{Z}$.*

4 Improved CMC Lower Bound of TdScript

While our primary technical contributions appear in [Section 3](#), for completeness, we verify that the results from [Section 3](#) imply the improved CMC lower bound for TdScript which is $\Omega(n^2 \log N)$ (see [Theorem 1](#)). The remainder of the arguments closely follow prior work [AGP24], but for completeness, we sketch the argument and highlight the modifications that lead to the improved bound.

4.1 Single-Challenge Tradeoff

Now, we can improve [AGP24, Lemma 1] to [Lemma 4](#). Here, $\text{params}(\lambda)$ denotes the set of all possible parameters of the form (N, Σ, H, w) where N is the group order, Σ denotes the labeling map, H is a random oracle, and w is a discrete log of $W \in \text{QR}_{N'}$. And TimeSC refers to the earliest round $i \in \mathbb{N}$ in which $\mathcal{A}_{\text{on}}^{\mathcal{G}, H}(\text{str}, j)$ queries $\Sigma(w2^j)$ to H (or ∞ if it never does). To ensure this paper is self-contained, we cite [Definition 4](#) and [Definition 5](#) quoted verbatim from [AGP24].

Definition 4 ([AGP24, Definition 5]). For security parameter $\lambda \in \mathbb{N}$, let $\text{params}(\lambda)$ be the set of all possible parameters. It contains all quadruples (N, Σ, H, w) where $(p', q') \leftarrow \text{GenSP}(\lambda)$ with $p := (p' - 1)/2$ and $q := (q' - 1)/2$ defines the group order $N := pq$, the labeling function Σ is an injection from $\mathbb{Z}_N$ to $\mathcal{L} = \{0, 1\}^{\omega_{\mathcal{L}}}$, $H : \{0, 1\}^* \rightarrow \{0, 1\}^{\omega_H}$ is a random oracle, and the input w is non-zero, i.e., $w \in \mathbb{Z}_N \setminus \{0\}$. Here, w is the discrete logarithm of W , i.e., $\Sigma^{-1}(W) = w$ in the GGM. Furthermore, recall that we require $\omega_{\mathcal{L}} \geq \lceil \log N \rceil$ and $\omega_H \in \Theta(\log N)$.

To avoid clutter, we sometimes leave λ implicit and write params instead of $\text{params}(\lambda)$.

Definition 5 ([AGP24, Definition 6]). Let $(N, \Sigma, H, w) \in \text{params}$ and $\mathcal{A} = (\mathcal{A}_{\text{pre}}, \mathcal{A}_{\text{on}})$ be deterministic parallel oracle machines. To the challenge $j \in [0, n)$ and state $\text{st}_0 := \mathcal{A}_{\text{pre}}^{\Sigma, H}(\Sigma(1), \Sigma(w))$ we associate

$$\text{TimeSC}_{\mathcal{A}_{\text{on}}}^{\Sigma, H}(\text{st}_0, j) := \begin{cases} \min \{i : \Sigma(w2^j) \in \mathbf{qrs}_i^H\} & \text{if the minimum exists, and} \\ \infty & \text{otherwise.} \end{cases}$$

So TimeSC is the earliest round $i \in \mathbb{N}$ in which $\mathcal{A}_{\text{on}}^{\Sigma, H}(\text{st}_0, j)$ queries $\Sigma(w2^j)$ to H or ∞ if it never does.

Now we present our improved **Lemma 4** which characterizes the set bad . Again, the statement of **Lemma 4** looks similar to [AGP24, Lemma 1] except that the constraint on TimeSC is improved from $n/(6\rho \log(n/2))$ to $n/(4e\rho)$, eliminating the $\log(n/2)$ term.

Lemma 4 (Single-Challenge Tradeoff). *For every pair of deterministic parallel oracle machines $\mathcal{A} = (\mathcal{A}_{\text{pre}}, \mathcal{A}_{\text{on}})$, all $c \in \mathbb{N}$ with security parameter λ large enough, and all $n, M, Q \in \text{poly}(\lambda)$ with $n \geq 8$ and subsets $\text{bad} \subseteq \text{params}(\lambda)$, if, for every $(N, \Sigma, H, w) \in \text{bad}$, $\mathcal{A}_{\text{on}}$ makes at most Q queries, $\text{st}_0 := \mathcal{A}_{\text{pre}}^{\Sigma, H}(\Sigma(1), \Sigma(w))$ is of size $|\text{st}_0| \leq M$ and*

$$\Pr_{j \leftarrow \mathbb{s}[0, n)} \left[\text{TimeSC}_{\mathcal{A}_{\text{on}}}^{\Sigma, H}(\text{st}_0, j) \leq \frac{n}{4e\rho} \right] \geq \frac{1}{2},$$

where $\rho = (M + \log n + \log Q + c \log \lambda + 1)/(\log N - 3(\log Q - \log n) - 3)$, then $|\text{bad}| < \lambda^{-c} |\text{params}(\lambda)|$.

The proof of **Lemma 4** is largely the same as the proof of [AGP24, Lemma 1], except that the $n/(6\rho \log(n/2))$ part is updated to $n/(4e\rho)$, which simplifies the case analysis. In particular, [AGP24, Claim 2] can be updated to **Claim 6**.

Claim 6. *If Lemma 4 does not hold and $\rho \leq n/(4e)$, for all $(N, \Sigma, H, w) \in \text{bad}$, $\text{rank}(A) \geq \rho$.*

Proof. By the assumption that **Lemma 4** does not hold, $\mathcal{A}_{\text{on}}$ answers at least $\ell := n/2 \geq 4$ challenges within $t := n/(4e\rho) - 1$ rounds of queries. By [AGP24, Claim 1], we may assume that bad only contains parameters where the system $A\mathbf{x} = \mathbf{b}$ has a solution. Hence, we can apply **Lemma 2** to get

$$\text{rank}(A) \geq \frac{n}{4et} \geq \frac{n}{4e} \cdot \frac{4e\rho}{n} = \rho. \quad \square$$

Proof of Lemma 4. We consider the following two cases.

Case $\rho > n/(4e)$: Then $n/(4e\rho) < 1$ and the tradeoff holds for any $j \in [0, n)$ since $\text{TimeSC} \geq 1$ by [AGP24, Definition 6].

Case $\rho \leq n/(4e)$: Towards contradiction, suppose that **Lemma 4** does not hold. Then by **Claim 6**, we have $\text{rank}(A) \geq \rho$. However, [AGP24, Claim 3] tells us that if **Lemma 4** does not hold then there exists a tuple $(N, \Sigma, H, w) \in \text{bad}$ such that $\text{rank}(A) < \rho$ (the same proof carries over when we update the number of rounds $t = \frac{n}{6\rho \log(n/2)} - 1$ in the proof of [AGP24, Claim 5] to $t = \frac{n}{4e\rho} - 1$), which contradicts each other. Hence, **Lemma 4** holds. $\square$

While Lemma 4 characterizes the set `bad` in which the adversary answers the challenge more quickly than expected, one can characterize the set `good` (the other way around) as stated below in Corollary 1 which is similar to [AGP24, Corollary 1] except that the constraint on TimeSC is improved from $n/(6\rho \log(n/2))$ to $n/(4e\rho)$.

Corollary 1. *For every pair of deterministic parallel oracle machines $\mathcal{A} = (\mathcal{A}_{\text{pre}}, \mathcal{A}_{\text{on}})$, there exists a negligible function $\epsilon(\lambda) \in \text{negl}(\lambda)$ such that, for all $n, M, Q \in \text{poly}(\lambda)$ with $n \geq 8$, there exists a subset $\text{good} \subseteq \text{params}(\lambda)$ such that, for every $(N, \Sigma, H, w) \in \text{good}$, $\mathcal{A}_{\text{on}}$ makes at most Q queries, $\text{st}_0 := \mathcal{A}_{\text{pre}}^{\Sigma, H}(\Sigma(1), \Sigma(w))$ is of size $|\text{st}_0| \leq M$,*

$$\Pr_{j \leftarrow \mathbb{S}[0, n)} \left[\text{TimeSC}_{\mathcal{A}_{\text{on}}}^{\Sigma, H}(\text{st}_0, j) > \frac{n}{4e\rho} \right] \geq \frac{1}{2},$$

where $\rho \in \Theta((M + \log n + \log Q + c \log \lambda + 1)/(\log N - 3(\log Q - \log n) - 3))$ and $|\text{good}| \geq (1 - \epsilon(\lambda))|\text{params}(\lambda)|$.

4.2 Improved Multi-Challenge Memory Complexity

Applying the improvements that we achieved, the CMC lower bound of TdSCRYPT [AGP24, Theorem 1] can be improved as follows.

Theorem 1. *Let $n \in \text{poly}(\lambda)$ with $n \geq 8$ and let $\mathcal{A}$ be a deterministic parallel oracle machine that evaluates TdSCRYPT correctly with probability $\chi(\lambda)$ over the choice of the parameters and input W . Then, assuming that the factoring is hard, in the GGM and ROM with probability at least $\chi(\lambda) - \epsilon(\lambda)$,*

$$\text{cmc}(\mathcal{A}^{\Sigma, H}(\Sigma(1), \Sigma(w))) \in \Omega(n^2 \log N),$$

where $\epsilon(\lambda) \in \text{negl}(\lambda)$, and the probability is taken over $(N, \Sigma, H, w) \leftarrow \mathbb{S} \text{ params}(\lambda)$.

For the rest of Section 4.2, we verify that our improved Theorem 1 holds by applying our results from Section 3 and Section 4.1.

Review: Proof Strategy of Auerbach et al. Auerbach et al. [AGP24] proved [AGP24, Theorem 1] by adapting [AGP24, Corollary 1] (which is similar to Corollary 1 but with the constraint on TimeSC set to $n/(6\rho \log(n/2))$ instead of $n/(4e\rho)$) to a single adversary $\mathcal{A}$, potentially viewed as a dishonest evaluation strategy for TdSCRYPT. For some $k \in [n]$, they split $\mathcal{A}$'s execution into two phases which correspond to $\mathcal{A}_{\text{pre}}$ and $\mathcal{A}_{\text{on}}$ in [AGP24, Corollary 1], respectively. Intuitively, the phases are divided at the round s_k where the k^{th} challenge $j_k = S_{k-1} \bmod n$ is issued. The challenge index j in $\mathcal{A}_{\text{on}}$ is simulated by programming the random oracle to ensure $S_{k-1} \bmod n = j$ while preserving $\lfloor S_{k-1}/n \rfloor$. Consequently, the hint st_0 for $\mathcal{A}_{\text{on}}$ is taken as the input state st_{s_k} . To establish a time-memory tradeoff, they showed that for at least half of the n possible values of j_k , the adversary must either invest a substantial number of rounds to answer the challenge or ensure that the size of stored states $\|\text{st}_r\|$ grow and remain significantly large as r approaches s_k . By applying Hoeffding's inequality, this tradeoff extends to roughly half of all n challenges, allowing the summation of state sizes across all challenges to derive the CMC lower bound.

Furthermore, Auerbach et al. [AGP24] defined three bad events, called `collidingk`, `roundingk(n)`, and `wrongOrderk`, where the time-memory tradeoff illustrated above might fail (see Definition 6). Intuitively, `collidingk` is the event where we cannot program $j_k = S_{k-1} \bmod n$ due to a collision with a preceding S_i for some i ; `roundingk(n)` describes the event that rounding issues, such as overflow of S_{k-1} , prevent the proper programming of j_k ; and `wrongOrderk` happens when the adversary queries the random oracle with $(\cdot, S_j)$ before $(\cdot, S_i)$ for some $0 \leq i < j < k$, i.e., the adversary learns the values $(S_i)_{0 \leq i \leq n}$ not in a sequential order.

Definition 6 ([AGP24, Definition 9,10,11]). For given N, Σ , and w , we define the following sets.

- (Collisions) The set colliding_k contains all random oracles H that cause a collision amongst the preceding $\{S_0, \dots, S_{k-1}\}$ during the honest $\text{Eval}(\mathbf{p}, \Sigma(w))$.
- (Rounding Impossible) The set $\text{rounding}_k(n)$ contains all random oracles H that are not amenable to programming some challenge $j \in [0, n)$. That is, there exists $0 \leq i \leq k$ such that $S_i > \lfloor 2^\ell / n \rfloor n - 1$.
- (Wrong Evaluation Order) For $\mathcal{A}$, the set wrongOrder_k contains all random oracles H where during the execution of $\mathcal{A}^{\Sigma, H}(\Sigma(w))$ a query to H of the form $(\cdot, S_j)$ occurs before one of the form $(\cdot, S_i)$ for $0 \leq i < j < k$.

They then defined a “hard challenge” where it takes more than a certain amount of time to answer the challenge [AGP24, Definition 12], and showed that, for most parameter choices, the instances do not fall into the three possible bad sets (i.e., colliding_k , $\text{rounding}_k(n)$, and wrongOrder_k), ensuring the time-memory tradeoff holds [AGP24, Lemma 5]. And then they applied this argument to lower bound $\sum_{r=1}^{s_n+1} \rho_r$ [AGP24, Claim 6], with $M_r = \|\text{st}_r\| \in \Omega(\rho_r \log N)$ implying the CMC lower bound for TdSCRIPT.

Improvements upon Prior Work. Since we have an improved time constraint from Lemma 4, we have an updated definition for hard challenges as shown in Definition 7. Note that in Definition 7, everything remains the same as [AGP24, Definition 12] except that the definition of r_k and $\text{Hard}_{\mathcal{A}(w)}^{\Sigma, H, n}(k, j)$ is changed due to our improved time constraint in Lemma 4 (from $n/(6\rho \log(n/2))$ to $n/(4e\rho)$).

Definition 7 (Hard Challenge). Let $\lambda, n \in \mathbb{N}$ with $n \geq 8$, $k, j \in [0, n)$, $(N, \Sigma, H, w) \in \text{params}(\lambda)$ with $H \notin \text{colliding}_{k-1} \cup \text{rounding}_{k-1}(n)$, and $\mathcal{A}$ a deterministic oracle machine making at most $Q \in \text{poly}(\lambda)$ queries. Consider the execution $\mathcal{A}^{\Sigma, H}(\Sigma(1), \Sigma(w))$ up to round $s_k \in \mathbb{N}$, which is the earliest round where the query $(W^{2^{j_{k-1}}}, S_{k-2}) \in \text{qrs}^H$ with $j_{k-1} = S_{k-2} \bmod n$ occurs (in the case $k = 1$, the query is $(W^{2^n}, 0^\ell)$). For any round r with $0 \leq r \leq s_k$, let $M_r = \|\text{st}_r\|$, and associate a fixed value ρ_r to it similarly to Lemma 4.⁵ It follows that

$$\rho_r \in \Theta(M_r + \log n + \log Q + \log \lambda) / (\log N - \log Q - \log n).$$

Define $H' = H$ except for S_{k-1} which will be changed to S'_{k-1} with $\lfloor S'_{k-1}/n \rfloor = \lfloor S_{k-1}/n \rfloor$ but $S'_{k-1} \bmod n = j$, and consider the execution $\mathcal{A}^{\Sigma, H'}(\Sigma(1), \Sigma(w))$. Let $t_{k,j,r} \in \mathbb{N}$ be minimal such that the query $(W^{2^j}, S_{k-1}) \in \text{qrs}^{H'}$ occurs after s_k in round $r + t_{k,j,r} > s_k$ where $t_{k,j,r} = \infty$ if it never does. Then, let

$$r_k = \arg \max_{0 \leq r \leq s_k} \left\{ \frac{n}{4e\rho_r} - (s_k - r) \right\},$$

and define

$$\text{Hard}_{\mathcal{A}(w)}^{\Sigma, H, n}(k, j) = 1 \text{ if and only if } t_{k,j,r} > \frac{n}{4e\rho_{r_k}}.$$

Even though the definition of $\text{Hard}_{\mathcal{A}(w)}^{\Sigma, H, n}(k, j)$ was changed, Lemma 5 stays the same as prior work. For completeness, we cite the Lemma quoted verbatim.

⁵To be precise, $\rho_r = (M + \log n + 2 \log Q + c \log \lambda + 1) / (\log N - 3(\log Q - \log n) - 3)$ where c is as in Lemma 4.

Lemma 5 ([AGP24, Lemma 5]). *For every deterministic parallel oracle machine $\mathcal{A}$, there exist negligible functions $\epsilon(\lambda) \in \text{negl}(\lambda)$ such that, for all $n \in \text{poly}(\lambda)$ with $n \geq 8$, there exists a subset $\text{good} \subseteq \text{params}(\lambda)$ such that, for every $(N, \Sigma, H, w) \in \text{good}$ and $k \in [n]$, $|\text{good}| \geq (1 - \epsilon(\lambda))|\text{params}(\lambda)|$ and the following holds:*

- $H \notin \text{colliding}_{k-1}$;
- $H \notin \text{rounding}_{k-1}(n)$;
- $H \notin \text{wrongOrder}_{k-1}$; and
- $\Pr_{j \leftarrow \mathbb{S}[0, n]} [\text{Hard}_{\mathcal{A}(w)}^{\Sigma, H, n}(k, j) = 1] \geq \frac{1}{2}$.

Claim 7 is an improvement upon [AGP24, Claim 6]. We show that the lower bound if the parameters are considered good and the adversary queries S_n then the lower bound for $\sum \rho_r$ is $\Omega(n^2)$ which is a log n -factor improvement from the prior lower bound $\Omega(n^2 / \log n)$ [AGP24].

Claim 7 (Lower Bound on $\sum \rho_r$). *If $(N, \Sigma, H, w) \in \text{good}$ and $\mathcal{A}^{\Sigma, H}(\Sigma(1), \Sigma(w))$ queries S_n , then $\sum_{r=1}^{s_n+1} \rho_r \in \Omega(n^2)$.*

Proof Sketch. Since the proof largely works the same as prior work [AGP24], we will follow the same proof strategy and highlight the parts that are improved.

If parameters are in **good**, whenever $\mathcal{A}$ queries S_n , it must have received all $\{S_i\}_{0 \leq i \leq n}$ during its execution, and it must have received them in order. Since our definition of **Hard** is different from the prior work [AGP24], we observe that if $\text{Hard}_{\mathcal{A}(w)}^{\Sigma, H, n}(k, j) = 1$, then, for any $0 \leq r \leq s_k$, we have

$$t_{k,j,r} > \frac{n}{4e\rho_r}, \quad (4)$$

by the choice of r_k (see [ACP⁺17, Claims 5&10]). Finally, by a generalization of Hoeffding's inequality (cf. [ACP⁺17, Claim 7]), we can argue that for any fixing of challenges, at least $n(1/2 - \epsilon)$ challenges are **Hard** where $\epsilon > 0$ (cf. [ACP⁺17, Claim 19]).

Given the above, we can apply [ACP⁺17, Claim 8] to $\rho_r > \frac{n}{4et_{k,j,r}}$, which is Equation (4) rearranged. This gets us $\sum_{r=1}^{s_n+1} \rho_r \in \Omega(n^2)$ as desired (cf. [ACP⁺17, Claim 11]). $\square$

We are now ready to prove **Theorem 1** and confirm that the improvements from **Section 3** and **Section 4.1** extend to establish a tight CMC lower bound for **TdSCRYPT**.

Proof of Theorem 1 (Sketch). By **Lemma 5**, an overwhelming fraction of parameters is in **good**, and $\mathcal{A}$ queries S_n with probability $\chi(\lambda)$. Applying **Claim 7**, we get a lower bound of $\sum_{r=1}^{s_n+1} \rho_r \in \Omega(n^2)$. Since $M_r \in \Omega(\rho_r \log N)$ for all $r \in \mathbb{N}$ by definition, it follows that $\sum_{r=1}^{s_n+1} M_r \in \Omega(n^2 \log N)$. Note that this sum is equivalent to the **cmc** of $\mathcal{A}$. We also note that the proof idea largely works the same as prior work (see [AGP24, Theorem 1]) except that we improved the lower bound for $\sum_r \rho_r$ to $\Omega(n^2)$ (see **Claim 7**) which removed the $1/\log n$ factor from the prior lower bound $\Omega(n^2 / \log n)$ [AGP24, Claim 6]. $\square$

References

- [ABH17] Joël Alwen, Jeremiah Blocki, and Ben Harsha. Practical graphs for optimal side-channel resistant memory-hard functions. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017*, pages 1001–1017. ACM Press, October / November 2017. doi:10.1145/3133956.3134031.

- [ACP⁺17] Joël Alwen, Binyi Chen, Krzysztof Pietrzak, Leonid Reyzin, and Stefano Tessaro. Script is maximally memory-hard. In Jean-Sébastien Coron and Jesper Buus Nielsen, editors, *EUROCRYPT 2017, Part III*, volume 10212 of *LNCS*, pages 33–62. Springer, Cham, April / May 2017. doi:[10.1007/978-3-319-56617-7_2](https://doi.org/10.1007/978-3-319-56617-7_2).
- [AGP24] Benedikt Auerbach, Christoph U. Günther, and Krzysztof Pietrzak. Trapdoor memory-hard functions. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part III*, volume 14653 of *LNCS*, pages 315–344. Springer, Cham, May 2024. doi:[10.1007/978-3-031-58734-4_11](https://doi.org/10.1007/978-3-031-58734-4_11).
- [AS15] Joël Alwen and Vladimir Serbinenko. High parallel complexity graphs and memory-hard functions. In Rocco A. Servedio and Ronitt Rubinfeld, editors, *47th ACM STOC*, pages 595–603. ACM Press, June 2015. doi:[10.1145/2746539.2746622](https://doi.org/10.1145/2746539.2746622).
- [Bac02] Adam Back. Hashcash - A Denial of Service Counter-Measure. 09 2002.
- [BCS16] Dan Boneh, Henry Corrigan-Gibbs, and Stuart E. Schechter. Balloon hashing: A memory-hard function providing provable protection against sequential attacks. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *ASIACRYPT 2016, Part I*, volume 10031 of *LNCS*, pages 220–248. Springer, Berlin, Heidelberg, December 2016. doi:[10.1007/978-3-662-53887-6_8](https://doi.org/10.1007/978-3-662-53887-6_8).
- [BDK16] Alex Biryukov, Daniel Dinu, and Dmitry Khovratovich. Argon2: New generation of memory-hard functions for password hashing and other applications. In *IEEE European Symposium on Security and Privacy, EuroS&P 2016, Saarbrücken, Germany, March 21-24, 2016*, pages 292–302, 2016. URL: <http://dx.doi.org/10.1109/EuroSP.2016.31>, doi:[10.1109/EuroSP.2016.31](https://doi.org/10.1109/EuroSP.2016.31).
- [BH22] Jeremiah Blocki and Blake Holman. Sustained space and cumulative complexity trade-offs for data-dependent memory-hard functions. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part III*, volume 13509 of *LNCS*, pages 222–251. Springer, Cham, August 2022. doi:[10.1007/978-3-031-15982-4_8](https://doi.org/10.1007/978-3-031-15982-4_8).
- [BHK⁺19] Jeremiah Blocki, Benjamin Harsha, Siteng Kang, Seunghoon Lee, Lu Xing, and Samson Zhou. Data-independent memory hard functions: New attacks and stronger constructions. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part II*, volume 11693 of *LNCS*, pages 573–607. Springer, Cham, August 2019. doi:[10.1007/978-3-030-26951-7_20](https://doi.org/10.1007/978-3-030-26951-7_20).
- [BL22] Jeremiah Blocki and Seunghoon Lee. On the multi-user security of short schnorr signatures with preprocessing. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part II*, volume 13276 of *LNCS*, pages 614–643. Springer, Cham, May / June 2022. doi:[10.1007/978-3-031-07085-3_21](https://doi.org/10.1007/978-3-031-07085-3_21).
- [BLP23] Alex Biryukov and Marius Lombard-Platet. PURED: A unified framework for resource-hard functions. In Anupam Chattopadhyay, Shivam Bhasin, Stjepan Picek, and Chester Rebeiro, editors, *INDOCRYPT 2023, Part II*, volume 14460 of *LNCS*, pages 126–149. Springer, Cham, December 2023. doi:[10.1007/978-3-031-56235-8_7](https://doi.org/10.1007/978-3-031-56235-8_7).
- [BP17] Alex Biryukov and Léo Perrin. Symmetrically and asymmetrically hard cryptography. In Tsuyoshi Takagi and Thomas Peyrin, editors, *ASIACRYPT 2017, Part III*, volume 10626 of *LNCS*, pages 417–445. Springer, Cham, December 2017. doi:[10.1007/978-3-319-70700-6_15](https://doi.org/10.1007/978-3-319-70700-6_15).

- [BR93] Mihir Bellare and Phillip Rogaway. Random oracles are practical: A paradigm for designing efficient protocols. In Dorothy E. Denning, Raymond Pyle, Ravi Ganesan, Ravi S. Sandhu, and Victoria Ashby, editors, *ACM CCS 93*, pages 62–73. ACM Press, November 1993. doi:[10.1145/168588.168596](https://doi.org/10.1145/168588.168596).
- [CK18] Henry Corrigan-Gibbs and Dmitry Kogan. The discrete-logarithm problem with preprocessing. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part II*, volume 10821 of *LNCS*, pages 415–447. Springer, Cham, April / May 2018. doi:[10.1007/978-3-319-78375-8_14](https://doi.org/10.1007/978-3-319-78375-8_14).
- [DN93] Cynthia Dwork and Moni Naor. Pricing via processing or combatting junk mail. In Ernest F. Brickell, editor, *CRYPTO’92*, volume 740 of *LNCS*, pages 139–147. Springer, Berlin, Heidelberg, August 1993. doi:[10.1007/3-540-48071-4_10](https://doi.org/10.1007/3-540-48071-4_10).
- [DTT10] Anindya De, Luca Trevisan, and Madhur Tulsiani. Time space tradeoffs for attacks against one-way functions and PRGs. In Tal Rabin, editor, *CRYPTO 2010*, volume 6223 of *LNCS*, pages 649–665. Springer, Berlin, Heidelberg, August 2010. doi:[10.1007/978-3-642-14623-7_35](https://doi.org/10.1007/978-3-642-14623-7_35).
- [FLW14] Christian Forler, Stefan Lucks, and Jakob Wenzel. Memory-demanding password scrambling. In Palash Sarkar and Tetsu Iwata, editors, *ASIACRYPT 2014, Part II*, volume 8874 of *LNCS*, pages 289–305. Springer, Berlin, Heidelberg, December 2014. doi:[10.1007/978-3-662-45608-8_16](https://doi.org/10.1007/978-3-662-45608-8_16).
- [Hun20] Troy Hunt. Have i been pwned: Check if your email has been compromised in a data breach. URL: <https://haveibeenpwned.com>, 2020.
- [KMP16] Eike Kiltz, Daniel Masny, and Jiaxin Pan. Optimal security proofs for signatures from identification schemes. In Matthew Robshaw and Jonathan Katz, editors, *CRYPTO 2016, Part II*, volume 9815 of *LNCS*, pages 33–61. Springer, Berlin, Heidelberg, August 2016. doi:[10.1007/978-3-662-53008-5_2](https://doi.org/10.1007/978-3-662-53008-5_2).
- [LPA⁺19] Lucy Li, Bijeeta Pal, Junade Ali, Nick Sullivan, Rahul Chatterjee, and Thomas Ristenpart. Protocols for checking compromised credentials. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019*, pages 1387–1403. ACM Press, November 2019. doi:[10.1145/3319535.3354229](https://doi.org/10.1145/3319535.3354229).
- [Mau05] Ueli Maurer. Abstract models of computation in cryptography. In *Proceedings of the 10th International Conference on Cryptography and Coding*, IMA’05, page 1–12, Berlin, Heidelberg, 2005. Springer-Verlag. doi:[10.1007/11586821_1](https://doi.org/10.1007/11586821_1).
- [NBF⁺16] Arvind Narayanan, Joseph Bonneau, Edward Felten, Andrew Miller, and Steven Goldfeder. *Bitcoin and cryptocurrency technologies: a comprehensive introduction*. Princeton University Press, 2016.
- [Nec94] V. I. Nechaev. Complexity of a determinate algorithm for the discrete logarithm. *Math Notes*, 55:165, 1994. doi:[10.1007/BF02113297](https://doi.org/10.1007/BF02113297).
- [Per09] Colin Percival. Stronger key derivation via sequential memory-hard functions. 01 2009.
- [PTB19] Jennifer Pullman, Kurt Thomas, and Elie Bursztein. Protect your accounts from data breaches with password checkup. Retrieved September, 4:2024, 2019.

- [RD17] Ling Ren and Srinivas Devadas. Bandwidth hard functions for ASIC resistance. In Yael Kalai and Leonid Reyzin, editors, *TCC 2017, Part I*, volume 10677 of *LNCS*, pages 466–492. Springer, Cham, November 2017. doi:[10.1007/978-3-319-70500-2_16](https://doi.org/10.1007/978-3-319-70500-2_16).
- [Sho97] Victor Shoup. Lower bounds for discrete logarithms and related problems. In Walter Fumy, editor, *EUROCRYPT'97*, volume 1233 of *LNCS*, pages 256–266. Springer, Berlin, Heidelberg, May 1997. doi:[10.1007/3-540-69053-0_18](https://doi.org/10.1007/3-540-69053-0_18).
- [SJ00] Claus-Peter Schnorr and Markus Jakobsson. Security of signed ElGamal encryption. In Tatsuaki Okamoto, editor, *ASIACRYPT 2000*, volume 1976 of *LNCS*, pages 73–89. Springer, Berlin, Heidelberg, December 2000. doi:[10.1007/3-540-44448-3_7](https://doi.org/10.1007/3-540-44448-3_7).
- [TPY⁺19] Kurt Thomas, Jennifer Pullman, Kevin Yeo, Ananth Raghunathan, Patrick Gage Kelley, Luca Invernizzi, Borbala Benko, Tadek Pietraszek, Sarvar Patel, Dan Boneh, and Elie Bursztein. Protecting accounts from credential stuffing with password breach alerting. In Nadia Heninger and Patrick Traynor, editors, *USENIX Security 2019*, pages 1556–1571. USENIX Association, August 2019.
- [Zha22] Mark Zhandry. To label, or not to label (in generic groups). In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part III*, volume 13509 of *LNCS*, pages 66–96. Springer, Cham, August 2022. doi:[10.1007/978-3-031-15982-4_3](https://doi.org/10.1007/978-3-031-15982-4_3).